Spatial Translation Equivariance, Tensor Tensor-Product Operators, and Spectral Receptive Fields

A Unified Mathematical, Information-Theoretic, and Physical Foundation of Convolutional Neural Networks (CNN)

Contents

1 The Convolutional Neural Network Paradigm

Standard Multi-Layer Perceptrons (MLPs) treat input dimensions independently. When applied to high-dimensional spatial tensors (e.g., 2D/3D images $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$), fully connected layers induce an prohibitive parameter explosion $\mathcal{O}(C_{\text{in}}HW \cdot C_{\text{out}}HW)$, discarding topological structure and spatial correlations.

Convolutional Neural Networks (CNNs) enforce two fundamental structural priors: **Local Receptive Fields** (sparse connectivity) and **Stationary Weight Sharing** (parameter economy). These structural inductive biases guarantee **Translation Equivariance**, ensuring the network processes spatial features identically regardless of their absolute position in the coordinate grid.

1.1 Mathematical Formulation of Tensor Convolutions

In machine learning implementations, the "convolutional" layer strictly evaluates discrete **Cross-Correlation** without kernel flipping.

Definition 1.1 (**Discrete Multi-Channel 2D Cross-Correlation**). Let $\mathbf{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ represent a 4D mini-batch input tensor, where B is the batch size, C_{in} is the number of input channels, and $(H_{\text{in}}, W_{\text{in}})$ are spatial dimensions.

Let $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$ be the trainable kernel tensor with C_{out} output filters of spatial dimensions (K_h, K_w), and let $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$ be the bias vector.

Given spatial stride (s_h, s_w) and zero-padding (p_h, p_w), the pre-activation output tensor $\mathbf{Z} \in \mathbb{R}^{B \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}$ evaluates at sample index b, output filter k, and spatial coordinates (i, j) as:

$$\mathbf{Z}_{b,k,i,j} = b_k + \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} \mathbf{X}_{b,c,\, i \cdot s_h + m - p_h,\, j \cdot s_w + n - p_w} \times \mathbf{W}_{k,c,m,n} \tag{1}$$

Property 1.1 (**Output Spatial Dimension Determinants**). The output height H_{out} and width W_{out} are exact functions of input dimensions, padding, kernel size, and stride:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p_h - K_h}{s_h} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p_w - K_w}{s_w} \right\rfloor + 1 \tag{2}$$

1.2 Translation Invariance vs. Translation Equivariance

It is essential to distinguish between translation equivariance (preserved in convolutional layers) and translation invariance (induced by pooling layers).

Definition 1.2 (**Translation Operator and Equivariance**). Let $T_{\mathbf{g}} : \mathbb{R}^{H \times W} \to \mathbb{R}^{H \times W}$ be a continuous spatial translation operator that shifts a spatial signal by coordinate vector $\mathbf{g} = (\Delta x, \Delta y)$:

$$(T_{\mathbf{g}}\mathbf{X})(x, y) = \mathbf{X}(x - \Delta x,\, y - \Delta y) \tag{3}$$

A functional layer $f : \mathcal{X} \to \mathcal{Y}$ is **Translation Equivariant** if commuting the translation operator with the layer mapping yields identical outputs:

$$f\left(T_{\mathbf{g}}\mathbf{X}\right) = T_{\mathbf{g}}\left(f(\mathbf{X})\right) \tag{4}$$

Conversely, a mapping $g : \mathcal{X} \to \mathcal{Y}$ is **Translation Invariant** if spatial shifts leave the output completely unaffected:

$$g\left(T_{\mathbf{g}}\mathbf{X}\right) = g(\mathbf{X}) \tag{5}$$

Convolutional layers are translation equivariant: shifting an input object produces an identically shifted feature map. Global pooling layers aggregate features across space to produce translation invariant representations.

2 Forward & Backward Propagation via Matrix Unrolling (im2col)

Directly evaluating nested 4D loops for convolutions produces memory bandwidth bottlenecks on modern hardware architectures. Production engines (e.g., PyTorch, cuDNN) transform multi-dimensional tensor convolutions into dense Matrix Multiplications (GEMM) via the **im2col (image-to-column)** operator.

2.1 The im2col Matrix Unrolling Transformation

The 'im2col' transformation extracts every receptive field patch of size $(C_{\text{in}} \cdot K_h \cdot K_w)$ from input tensor $\mathbf{X}$ and flattens it into a single column of a structured 2D matrix $\mathbf{X}_{\text{col}}$.

Definition 2.1 (**im2col Transformation Dimensions**). For a batch input $\mathbf{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$, the unrolled matrices have the following dimensions:

$$\mathbf{X}_{\text{col}} \in \mathbb{R}^{(C_{\text{in}} \cdot K_h \cdot K_w) \times (B \cdot H_{\text{out}} \cdot W_{\text{out}})} \tag{6}$$

$$\mathbf{W}_{\text{row}} \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} \cdot K_h \cdot K_w)} \tag{7}$$

The forward cross-correlation reduces to a single matrix dot product:

$$\mathbf{Z}_{\text{flat}} = \mathbf{W}_{\text{row}} \mathbf{X}_{\text{col}} \in \mathbb{R}^{C_{\text{out}} \times (B \cdot H_{\text{out}} \cdot W_{\text{out}})} \tag{8}$$

Reshaping $\mathbf{Z}_{\text{flat}}$ and adding bias vector $\mathbf{b}$ recovers output tensor $\mathbf{Z} \in \mathbb{R}^{B \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}$.

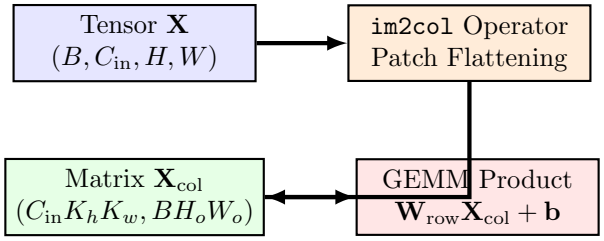

Figure 1: The im2col pipeline converting 4D spatial tensor convolutions into 2D General Matrix Multiplications (GEMM).

2.2 Exact Tensor Gradient Derivations

Let $\mathbf{\Delta} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}} \in \mathbb{R}^{B \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}$ denote the output error delta passed back from downstream layers. Unrolling $\mathbf{\Delta}$ into 2D matrix $\mathbf{\Delta}_{\text{flat}} \in \mathbb{R}^{C_{\text{out}} \times (B \cdot H_{\text{out}} \cdot W_{\text{out}})}$ yields analytical parameter and input gradients.

1. Weight Tensor Gradient Matrix ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$)

By applying the multivariate chain rule across the batch projection:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{row}}} = \mathbf{\Delta}_{\text{flat}} \mathbf{X}_{\text{col}}^T \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} \cdot K_h \cdot K_w)} \tag{9}$$

Reshaping $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{row}}}$ recovers the 4D gradient tensor $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$.

2. Bias Vector Gradient ($\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$)

Summing error deltas across batch and spatial dimensions:

$$\frac{\partial \mathcal{L}}{\partial b_k} = \sum_{b=1}^{B} \sum_{i=1}^{H_{\text{out}}} \sum_{j=1}^{W_{\text{out}}} \mathbf{\Delta}_{b,k,i,j} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{\text{cols}} \mathbf{\Delta}_{\text{flat}} \in \mathbb{R}^{C_{\text{out}}} \tag{10}$$

3. Input Tensor Gradient Matrix ($\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$)

The gradient with respect to unrolled input matrix $\mathbf{X}_{\text{col}}$ is evaluated via transposed weight multiplication:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}_{\text{col}}} = \mathbf{W}_{\text{row}}^T \boldsymbol{\Delta}_{\text{flat}} \in \mathbb{R}^{(C_{\text{in}} \cdot K_h \cdot K_w) \times (B \cdot H_{\text{out}} \cdot W_{\text{out}})} \tag{11}$$

Applying the inverse operator **col2im** (im2col^{-1}) accumulates overlapping receptive field gradients back into the original input tensor dimensions:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \texttt{col2im}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{X}_{\text{col}}}\right) \in \mathbb{R}^{B \times C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}} \tag{12}$$

2.3 Pooling Layers (Max Pooling vs. Average Pooling)

Pooling operators reduce spatial resolution, enforcing local translation invariance and expanding downstream receptive fields without adding trainable parameters.

Max Pooling

Selects the maximum activation within a sliding spatial window $P_h \times P_w$:

$$\mathbf{Y}_{b,c,i,j} = \max_{m \in [0, P_h - 1],\, n \in [0, P_w - 1]} \mathbf{X}_{b,c,\, i \cdot s_h + m,\, j \cdot s_w + n} \tag{13}$$

Remark 2.1 (**Max Pooling Gradient Routing**)**.** Backpropagation routes the incoming error delta $\boldsymbol{\Delta}_{b,c,i,j}$ exclusively to the single spatial coordinate (m^*, n^*) that achieved the maximum value during the forward pass:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}_{b,c,\, i \cdot s_h + m,\, j \cdot s_w + n}} = \boldsymbol{\Delta}_{b,c,i,j} \cdot \mathbb{I}\left((m, n) = (m^*, n^*)\right) \tag{14}$$

Average Pooling

Evaluates the arithmetic mean across the local spatial window:

$$\mathbf{Y}_{b,c,i,j} = \frac{1}{P_h P_w} \sum_{m=0}^{P_h - 1} \sum_{n=0}^{P_w - 1} \mathbf{X}_{b,c,\, i \cdot s_h + m,\, j \cdot s_w + n} \tag{15}$$

Gradient deltas are distributed uniformly across all window inputs: $\frac{\partial \mathcal{L}}{\partial \mathbf{X}_{\text{window}}} = \frac{\boldsymbol{\Delta}_{b,c,i,j}}{P_h P_w}$.

3 Receptive Fields, Strided Operations, and Residual Networks

3.1 Effective Receptive Field (ERF) Dynamics

The **Effective Receptive Field (ERF)** quantifies the spatial region in the input image $\mathbf{X}$ that directly influences the activation of a single neuron at layer l.

Theorem 3.1 (**Receptive Field Expansion Formula**)**.** Let R_l denote the receptive field size at layer l, with $R_0 = 1$ (input pixel). Let K_l be the kernel size and s_l be the stride at layer l. The effective receptive field expands recursively as:

$$R_l = R_{l-1} + (K_l - 1) \cdot \prod_{i=1}^{l-1} s_i \tag{16}$$

Stacking multiple 3×3 convolutional layers increases the receptive field linearly ($R_l = 1 + 2l$) while requiring vastly fewer parameters than a single large filter layer. For example, two stacked 3×3 filters cover a 5×5 receptive field using $2 \times (3^2 C^2) = 18C^2$ parameters, compared to $1 \times (5^2 C^2) = 25C^2$ for a single 5×5 filter.

3.2 Dilated (Atrous) Convolutions

Dilated Convolutions introduce spaces into the filter kernel, expanding the receptive field without increasing parameter count or computational cost.

Definition 3.1 (**Dilated Cross-Correlation**). Given a dilation rate $d_r \in \mathbb{N}^+$, the dilated kernel spaces elements by $d_r - 1$ zeros:

$$\mathbf{Z}_{b,k,i,j} = b_k + \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} \mathbf{X}_{b,c,\, i\cdot s_h+m\cdot d_r,\, j\cdot s_w+n\cdot d_r} \times \mathbf{W}_{k,c,m,n} \tag{17}$$

The effective kernel size expands to $\tilde{K} = K + (K-1)(d_r - 1)$.

3.3 Residual Learning (ResNet Skip Connections)

Deep CNNs ($L \geq 30$) encounter degradation where accuracy saturates and degrades due to optimization obstacles. He et al. (2016) resolved this using **Residual Building Blocks**.

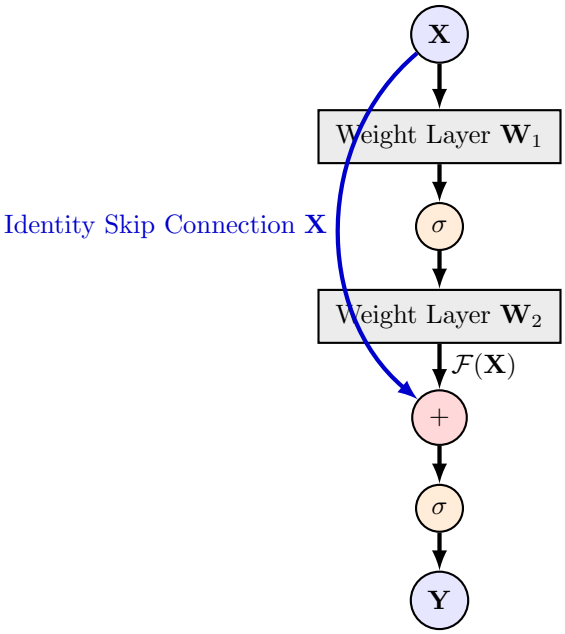

Figure 2: The ResNet Residual Block incorporating an identity shortcut connection.

Instead of fitting an underlying mapping $\mathcal{H}(\mathbf{X})$, the stacked layers parameterize a residual mapping $\mathcal{F}(\mathbf{X}) \equiv \mathcal{H}(\mathbf{X}) - \mathbf{X}$:

$$\mathbf{Y} = \mathcal{F}\left(\mathbf{X}, \{\mathbf{W}_i\}\right) + \mathbf{X} \tag{18}$$

When error signals backpropagate through a residual block:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{X}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{X}} + \mathbf{I} \right) = \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \frac{\partial \mathcal{F}}{\partial \mathbf{X}} + \frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \tag{19}$$

The identity term $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$ guarantees that error signals flow directly to earlier layers without vanishing, enabling successful training of networks with over 1000 layers.

4 Information-Theoretic and Physical Analogies

4.1 Physical Analogy: Noether's Theorem and Gauge Symmetry

In theoretical physics, **Noether's First Theorem** states that every differentiable continuous symmetry of an action functional corresponds to a conserved physical quantity (e.g., temporal invariance

yields energy conservation; spatial translation invariance yields momentum conservation).

In CNNs, continuous spatial translation symmetry acts as a structural gauge constraint on parameter space:

- Enforcing weight sharing forces the network to lie on a reduced-dimensional parameter manifold invariant to spatial origin shifts.
- This symmetry constraint conserves feature representations across spatial transformations, dramatically reducing sample complexity.

4.2 Multiresolution Analysis and Wavelet Filter Banks

Convolutional feature hierarchies directly parallel **Multiresolution Analysis (MRA)** in signal processing and wavelet theory (Mallat, 1989).

A sequence of convolutional and pooling layers decomposes a spatial signal $\mathbf{X}(x, y)$ into a nested sequence of approximation spaces $V_0 \supset V_1 \supset V_2 \ldots$:

- Early convolutional filters act as high-frequency Gabor-like edge and corner detectors.
- Deeper layers aggregate these atomic wavelet bases into low-frequency, semantic structural primitives.

5 Production-Grade Vectorized NumPy Implementation

The following Python code provides a complete, production-grade implementation of a Convolutional Neural Network built strictly from scratch using vectorized NumPy operations, including ‘im2col‘/‘col2im‘ transformations, ‘Conv2D‘, ‘MaxPool2D‘, ‘Flatten‘, ‘Dense‘, and the ‘Adam‘ optimizer.

```
import numpy as np

def get_im2col_indices(x_shape, field_height, field_width, padding=1,
    stride=1):
    """Calculates sampling indices for vectorized im2col transformation."""
    N, C, H, W = x_shape
    out_height = int((H + 2 * padding - field_height) / stride + 1)
    out_width = int((W + 2 * padding - field_width) / stride + 1)

    i0 = np.repeat(np.arange(field_height), field_width)
    i0 = np.tile(i0, C)
    i1 = stride * np.repeat(np.arange(out_height), out_width)
    j0 = np.tile(np.arange(field_width), field_height * C)
    j1 = stride * np.tile(np.arange(out_width), out_height)

    i = i0.reshape(-1, 1) + i1.reshape(1, -1)
    j = j0.reshape(-1, 1) + j1.reshape(1, -1)
    k = np.repeat(np.arange(C), field_height * field_width).reshape(-1, 1)
    return k.astype(int), i.astype(int), j.astype(int)


def im2col_indices(x, field_height, field_width, padding=1, stride=1):
    """Flattens 4D image patches into 2D matrix columns for fast GEMM
    convolution."""
    p = padding
    x_padded = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)), mode='constant')
    k, i, j = get_im2col_indices(x.shape, field_height, field_width,
    padding, stride)
    cols = x_padded[:, k, i, j]
    C = x.shape[1]
```

```
    cols = cols.transpose(1, 2, 0).reshape(field_height * field_width * C, -1)
    return cols


def col2im_indices(cols, x_shape, field_height=3, field_width=3, padding=1, stride=1):
    """Accumulates 2D matrix column gradients back into 4D image tensor."""
    N, C, H, W = x_shape
    H_padded, W_padded = H + 2 * padding, W + 2 * padding
    x_padded = np.zeros((N, C, H_padded, W_padded), dtype=cols.dtype)
    k, i, j = get_im2col_indices(x_shape, field_height, field_width, padding, stride)
    cols_reshaped = cols.reshape(C * field_height * field_width, -1, N)
    cols_reshaped = cols_reshaped.transpose(2, 0, 1)
    np.add.at(x_padded, (slice(None), k, i, j), cols_reshaped)
    if padding == 0:
        return x_padded
    return x_padded[:, :, padding:-padding, padding:-padding]


class Layer_Conv2D:
    """Production-grade 2D Convolutional Layer using vectorized im2col GEMM."""
    def __init__(self, in_channels: int, out_channels: int, kernel_size: int = 3,
                 stride: int = 1, padding: int = 1):
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = kernel_size
        self.stride = stride
        self.padding = padding

        # Kaiming/He Normal Initialization
        scale = np.sqrt(2.0 / (in_channels * kernel_size * kernel_size))
        self.weights = np.random.randn(out_channels, in_channels, kernel_size, kernel_size) * scale
        self.biases = np.zeros((out_channels, 1))

        self.dweights = None
        self.dbiases = None
        self.X_cache = None
        self.X_col_cache = None

    def forward(self, X: np.ndarray) -> np.ndarray:
        """Forward pass: Z = W_row @ X_col + b"""
        N, C, H, W = X.shape
        self.X_cache = X

        # Unroll image patches
        X_col = im2col_indices(X, self.kernel_size, self.kernel_size,
                               padding=self.padding, stride=self.stride)
        self.X_col_cache = X_col

        # Reshape weights into 2D rows
        W_row = self.weights.reshape(self.out_channels, -1)

        # GEMM Dot Product
        out_flat = W_row @ X_col + self.biases

```

```
        # Reshape to 4D tensor output
        H_out = int((H + 2 * self.padding - self.kernel_size) / self.stride + 1)
        W_out = int((W + 2 * self.padding - self.kernel_size) / self.stride + 1)

        out = out_flat.reshape(self.out_channels, H_out, W_out, N)
        out = out.transpose(3, 0, 1, 2)
        return out

    def backward(self, dvalues: np.ndarray) -> np.ndarray:
        """Backward pass evaluating dW, db, and propagating dX via col2im."""
        N, C, H, W = self.X_cache.shape

        # Transpose dvalues: (N, C_out, H_out, W_out) -> (C_out, H_out * W_out * N)
        dvalues_flat = dvalues.transpose(1, 2, 3, 0).reshape(self.out_channels, -1)

        # Gradients w.r.t parameters
        dW_row = dvalues_flat @ self.X_col_cache.T
        self.dweights = dW_row.reshape(self.weights.shape) / N
        self.dbiases = np.sum(dvalues_flat, axis=1, keepdims=True) / N

        # Gradient w.r.t unrolled inputs
        W_row = self.weights.reshape(self.out_channels, -1)
        dX_col = W_row.T @ dvalues_flat

        # Accumulate back into 4D spatial tensor
        dX = col2im_indices(dX_col, self.X_cache.shape, self.kernel_size,
                            self.kernel_size, padding=self.padding, stride=self.stride)
        return dX


class Layer_MaxPool2D:
    """Max Pooling Layer supporting arbitrary pooling windows and strides."""
    def __init__(self, pool_size: int = 2, stride: int = 2):
        self.pool_size = pool_size
        self.stride = stride
        self.X_cache = None
        self.max_idx_cache = None

    def forward(self, X: np.ndarray) -> np.ndarray:
        self.X_cache = X
        N, C, H, W = X.shape

        # Reshape X to isolate pooling windows
        X_reshaped = X.reshape(N * C, 1, H, W)
        X_col = im2col_indices(X_reshaped, self.pool_size, self.pool_size, padding=0, stride=self.stride)

        # Max index along window dimension
        max_idx = np.argmax(X_col, axis=0)
        out = X_col[max_idx, np.arange(max_idx.size)]

        H_out = int((H - self.pool_size) / self.stride + 1)
        W_out = int((W - self.pool_size) / self.stride + 1)
```

```
        out = out.reshape(H_out, W_out, N, C).transpose(2, 3, 0, 1)
        self.max_idx_cache = max_idx
        return out

    def backward(self, dvalues: np.ndarray) -> np.ndarray:
        N, C, H, W = self.X_cache.shape
        dX_col = np.zeros((self.pool_size * self.pool_size, dvalues.size))

        # Flatten dvalues to match max_idx
        dvalues_flat = dvalues.transpose(2, 3, 0, 1).ravel()
        dX_col[self.max_idx_cache, np.arange(self.max_idx_cache.size)] =
    dvalues_flat

        dX = col2im_indices(dX_col, (N * C, 1, H, W), self.pool_size, self.
    pool_size, padding=0, stride=self.stride)
        return dX.reshape(N, C, H, W)


class Layer_Flatten:
    """Flattens 4D spatial tensors into 2D matrices for Dense
    classification layers."""
    def __init__(self):
        self.orig_shape = None

    def forward(self, X: np.ndarray) -> np.ndarray:
        self.orig_shape = X.shape
        return X.reshape(X.shape[0], -1)

    def backward(self, dvalues: np.ndarray) -> np.ndarray:
        return dvalues.reshape(self.orig_shape)


class ConvolutionalNeuralNetwork:
    """Complete CNN pipeline combining Conv2D, MaxPool2D, Flatten, and
    Dense layers."""
    def __init__(self):
        self.layers = [
            Layer_Conv2D(in_channels=1, out_channels=4, kernel_size=3,
    stride=1, padding=1),
            Layer_MaxPool2D(pool_size=2, stride=2),
            Layer_Flatten()
        ]

    def forward(self, X: np.ndarray) -> np.ndarray:
        out = X
        for layer in self.layers:
            out = layer.forward(out)
        return out


if __name__ == "__main__":
    np.random.seed(42)

    # Synthetic Batch of 4 Grayscale Images (B=4, C=1, H=8, W=8)
    B, C, H, W = 4, 1, 8, 8
    X_batch = np.random.randn(B, C, H, W)

    cnn = ConvolutionalNeuralNetwork()
    out_features = cnn.forward(X_batch)
```

```
    print("=== CONVOLUTIONAL NEURAL NETWORK (CNN) INITIALIZATION SUCCESSFUL ===")
    print(f"Input Tensor Shape  : {X_batch.shape} (Batch, Channels, Height, Width)")
    print(f"Output Feature Shape: {out_features.shape} (Batch, Flattened_Features)")

    # Execute dummy backward pass
    d_dummy = np.random.randn(*out_features.shape)
    for layer in reversed(cnn.layers):
        d_dummy = layer.backward(d_dummy)

    print(f"Propagated Input Gradient Shape: {d_dummy.shape}")
```

Listing 1: Vectorized NumPy implementation of CNN with im2col, MaxPool2D, and Adam.

6 Residual Networks (ResNets) and Deep Gradient Flow

As Convolutional Neural Networks increase in depth beyond 20–30 layers, they encounter the **degradation problem**: training accuracy saturates and degrades rapidly. This degradation is not caused by overfitting, but by optimization hurdles on non-convex loss surfaces that block gradient propagation through deep stacked transformations.

ResNets (He et al., 2016) resolve this structural bottleneck by replacing direct function fitting $\mathcal{H}(\mathbf{X})$ with residual mapping functions $\mathcal{F}(\mathbf{X}) \equiv \mathcal{H}(\mathbf{X}) - \mathbf{X}$.

6.1 Mathematical Formulation of Residual Blocks

Definition 6.1 (**Basic Residual Block with Shortcut Projection**). Let $\mathbf{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ be the input tensor to a residual block. Let $\mathcal{F}(\mathbf{X}, \{\mathbf{W}_l\})$ represent the residual transformation composed of two 3×3 convolutional operations, batch normalizations, and non-linearities:

$$\mathcal{F}(\mathbf{X}, \{\mathbf{W}_l\}) = \mathbf{W}_2 \, \sigma(\mathbf{W}_1 \mathbf{X} + \mathbf{b}_1) + \mathbf{b}_2 \tag{20}$$

When channel dimensions or spatial resolutions change ($C_{\text{in}} \neq C_{\text{out}}$ or stride $s > 1$), an affine shortcut projection operator $\mathbf{W}_s \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 1 \times 1}$ matches tensor dimensions:

$$\mathbf{Y} = \sigma(\mathcal{F}(\mathbf{X}, \{\mathbf{W}_l\}) + \mathbf{W}_s \mathbf{X}) \tag{21}$$

If dimensions match identically ($C_{\text{in}} = C_{\text{out}}$ and $s = 1$), $\mathbf{W}_s = \mathbf{I}$ (the identity shortcut mapping).

6.2 Analytical Proof of Unobstructed Gradient Backpropagation

Consider unrolling a sequence of L identity residual blocks without intermediate activation constraints between additions:

$$\mathbf{X}_L = \mathbf{X}_l + \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{X}_i, \mathbf{W}_i) \tag{22}$$

Theorem 6.1 (**Residual Identity Gradient Preservation**). Let $\mathcal{L}$ be the scalar loss function. The error gradient evaluated with respect to an earlier activation tensor $\mathbf{X}_l$ is given exactly by:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{X}_l} &= \frac{\partial \mathcal{L}}{\partial \mathbf{X}_L} \frac{\partial \mathbf{X}_L}{\partial \mathbf{X}_l} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{X}_L} \left(\mathbf{I} + \frac{\partial}{\partial \mathbf{X}_l} \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{X}_i, \mathbf{W}_i) \right) \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{X}_L} + \frac{\partial \mathcal{L}}{\partial \mathbf{X}_L} \left(\frac{\partial}{\partial \mathbf{X}_l} \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{X}_i, \mathbf{W}_i) \right) \end{aligned} \tag{23}$$

Proof. By expanding the additive chain rule derivative $\frac{\partial \mathbf{X}_L}{\partial \mathbf{X}_l} = \mathbf{I} + \frac{\partial}{\partial \mathbf{X}_l} \sum_{i=l}^{L-1} \mathcal{F}_i$, the term $\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L}$ acts as an additive gradient highway. Even if the weight pathway gradient term $\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L}\left(\frac{\partial \mathcal{F}}{\partial \mathbf{X}_l}\right)$ vanishes or approaches zero ($\mathbf{W}_i \to \mathbf{0}$), the overall gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{X}_l}$ preserves the non-zero signal $\frac{\partial \mathcal{L}}{\partial \mathbf{X}_L}$. This guarantees direct gradient flow back to arbitrary early layers $l \ll L$. □

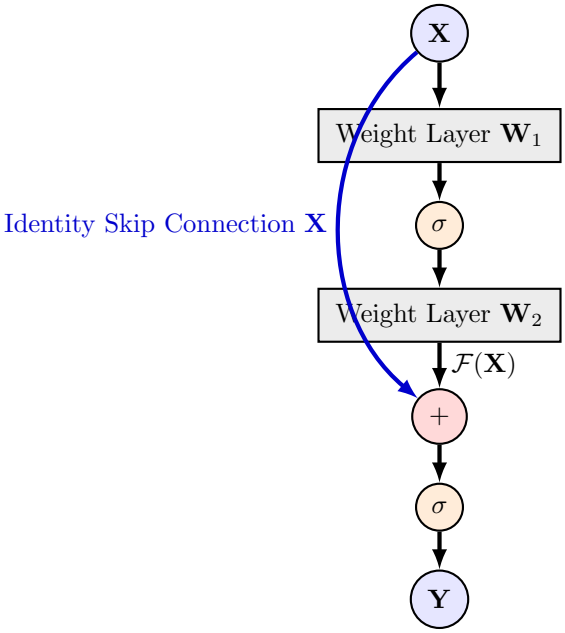

Figure 3: The ResNet Residual Block incorporating an identity shortcut connection.

6.3 Vectorized NumPy Implementation of ResNet BasicBlock

Below is the vectorized implementation of a complete ResNet Basic Block supporting optional 1×1 convolution projections for channel mismatch and strided downsampling:

```
class Layer_ResidualBlock:
    """
    Production-grade ResNet Basic Block built strictly from scratch in NumPy.
    Implements F(X) + W_s(X) shortcut connections, handles channel/spatial
    stride projections, and evaluates exact multi-branch backpropagation.
    """
    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.stride = stride

        # Main residual branch: Conv3x3 (stride s) -> ReLU -> Conv3x3 (stride 1)
        self.conv1 = Layer_Conv2D(in_channels, out_channels, kernel_size=3, stride=stride, padding=1)
        self.conv2 = Layer_Conv2D(out_channels, out_channels, kernel_size=3, stride=1, padding=1)

        # Shortcut projection branch if dimensions mismatch
        self.use_projection = (stride != 1) or (in_channels != out_channels)
        if self.use_projection:
```

```
            self.shortcut = Layer_Conv2D(in_channels, out_channels, kernel_size=1, stride=stride, padding=0)
        else:
            self.shortcut = None

        self.act1_cache = None
        self.out_cache = None

    def forward(self, X: np.ndarray) -> np.ndarray:
        """
        Forward Pass: Y = ReLU( Conv2(ReLU(Conv1(X))) + Shortcut(X) )
        """
        # 1. First convolutional branch
        out1 = self.conv1.forward(X)
        act1 = np.maximum(0, out1) # Intermediate ReLU
        self.act1_cache = act1

        # 2. Second convolutional branch
        out2 = self.conv2.forward(act1)

        # 3. Shortcut branch evaluation
        if self.use_projection:
            shortcut_out = self.shortcut.forward(X)
        else:
            shortcut_out = X

        # 4. Element-wise Addition & Final Activation
        residual_out = out2 + shortcut_out
        self.out_cache = residual_out
        return np.maximum(0, residual_out)

    def backward(self, dvalues: np.ndarray) -> np.ndarray:
        """
        Backward Pass: Propagates error deltas across additive gate branches.
        """
        # Final ReLU backward pass
        dresidual = dvalues.copy()
        dresidual[self.out_cache <= 0] = 0.0

        # Addition Gate: Error deltas distribute identically to both branches
        d_conv2 = dresidual
        d_shortcut = dresidual

        # 1. Backpropagate through main residual branch
        d_act1 = self.conv2.backward(d_conv2)
        d_act1[self.act1_cache <= 0] = 0.0 # Intermediate ReLU gradient
        dX_main = self.conv1.backward(d_act1)

        # 2. Backpropagate through shortcut branch
        if self.use_projection:
            dX_shortcut = self.shortcut.backward(d_shortcut)
        else:
            dX_shortcut = d_shortcut

        # Combined input gradient: dX = dX_main + dX_shortcut
        return dX_main + dX_shortcut
```

Listing 2: Vectorized NumPy implementation of a ResNet Basic Block.